VDJdb validation analytics

M.S.

2025-07-16

Load data

Load VDJdb table - here we use the version that matches the date of Messemaker et al. preprint release.

```
data <- read_tsv("dump/vdjdb-2025-02-21/vdjdb_full.txt") |>
  filter(species == "HomoSapiens") |>
  select(v.alpha, j.alpha, cdr3.alpha,
         v.beta, j.beta, cdr3.beta,
         antigen.epitope, mhc.a, mhc.b,
         method.identification,
         method.singlecell,
         reference.id) |>
  unique()
```

```
## Rows: 78826 Columns: 33
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (33): cdr3.alpha, v.alpha, j.alpha, cdr3.beta, v.beta, d.beta, j.beta, s...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
glimpse(data)
```

```
## Rows: 62,333
## Columns: 12
## $ v.alpha               <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ j.alpha               <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ cdr3.alpha            <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ v.beta                <chr> "TRBV19*01", "TRBV19*01", "TRBV19*01", "TRBV19*0~
## $ j.beta                <chr> "TRBJ2-1*01", "TRBJ2-2*01", "TRBJ2-3*01", "TRBJ2~
## $ cdr3.beta             <chr> "CASSIVGGNEQFF", "CASSMRSTGELFF", "CASSIRSAWAQYF~
## $ antigen.epitope       <chr> "GILGFVFTL", "GILGFVFTL", "GILGFVFTL", "GILGFVFT~
## $ mhc.a                 <chr> "HLA-A*02:01", "HLA-A*02:01", "HLA-A*02:01", "HL~
## $ mhc.b                 <chr> "B2M", "B2M", "B2M", "B2M", "B2M", "B2M", "B2M",~
## $ method.identification <chr> "tetramer-sort", "tetramer-sort", "tetramer-sort~
## $ method.singlecell     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ reference.id          <chr> "PMID:28629751", "PMID:28629751", "PMID:28629751~
```

```r
data |>
  filter(!is.na(cdr3.alpha)) |>
  nrow()
```

```
## [1] 40167
```

```r
data |>
  filter(!is.na(cdr3.beta)) |>
  nrow()
```

```
## [1] 54873
```

```r
data |>
  filter(!is.na(cdr3.alpha), !is.na(cdr3.beta)) |>
  nrow()
```

```
## [1] 32707
```

```r
length(unique(data$antigen.epitope))
```

```
## [1] 1266
```

Also load TCRNET motifs

```r
data.motifs <- read_tsv("dump/vdjdb-2025-02-21/cluster_members.txt") |>
  filter(species == "HomoSapiens")
```

```
## Rows: 8565 Columns: 19
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (14): species, antigen.epitope, antigen.gene, antigen.species, mhc.a, mh...
## dbl  (5): x, y, csz, v.end, j.start
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
```

```r
glimpse(data.motifs)
```

```
## Rows: 7,790
## Columns: 19
## $ species         <chr> "HomoSapiens", "HomoSapiens", "HomoSapiens", "HomoSapi~
## $ antigen.epitope <chr> "ATDALMTGF", "ATDALMTGF", "ATDALMTGF", "ATDALMTGF", "A~
## $ antigen.gene    <chr> "NS3", "NS3", "NS3", "NS3", "NS3", "NS3", "NS3", "NS3"~
## $ antigen.species <chr> "HCV", "HCV", "HCV", "HCV", "HCV", "HCV", "HCV", "HCV"~
## $ mhc.a           <chr> "HLA-A*01:01", "HLA-A*01:01", "HLA-A*01:01", "HLA-A*01~
## $ mhc.b           <chr> "B2M", "B2M", "B2M", "B2M", "B2M", "B2M", "B2M", "B2M"~
## $ mhc.class       <chr> "MHCI", "MHCI", "MHCI", "MHCI", "MHCI", "MHCI", "MHCI"~
## $ gene            <chr> "TRA", "TRA", "TRA", "TRA", "TRA", "TRA", "TRA", "TRA"~
## $ cdr3aa          <chr> "CAVFNTGGFKTIF", "CAVRGTGGFKTIF", "CAVRNTGGFKTIF", "CA~
```

```
## $ x                <dbl> 33.66516, 182.79251, 109.23737, 219.64220, 171.15401, ~
## $ y                <dbl> 670.9924, 719.6963, 686.7195, 604.5821, 664.6011, 736.~
## $ cid              <chr> "H.A.ATDALMTGF.5", "H.A.ATDALMTGF.5", "H.A.ATDALMTGF.5~
## $ csz              <dbl> 6, 6, 6, 6, 6, 6, 10, 10, 10, 10, 10, 10, 10, 10, 10, ~
## $ v.segm           <chr> "TRAV21*01", "TRAV21*01", "TRAV21*01", "TRAV21*01", "T~
## $ j.segm           <chr> "TRAJ9*01", "TRAJ9*01", "TRAJ9*01", "TRAJ9*01", "TRAJ9~
## $ v.end            <dbl> 3, 4, 4, 4, 4, 4, 4, 4, 4, 3, 3, 3, 3, 3, 3, 3, 3, 5, ~
## $ j.start          <dbl> 4, 5, 4, 6, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, ~
## $ v.segm.repr      <chr> "TRAV21*01", "TRAV21*01", "TRAV21*01", "TRAV21*01", "T~
## $ j.segm.repr      <chr> "TRAJ9*01", "TRAJ9*01", "TRAJ9*01", "TRAJ9*01", "TRAJ9~
```

Summary of antigen specificity database biases

Compare imbalance in terms of uneven HLA/epitope coverage. Also check how this imbalance drops when going from records to motifs. Note that we can compute and compare Gini indices which would be more statistically sound, however, for the sake of simplicity for the reader we just explore the case of top 1 by coverage vs median.

By MHC, we take only first chain focusing on MHCI, there are obviously far few MHCII

```r
data.s.mhc <- data |>
  mutate(mhc.a = str_split_fixed(mhc.a, fixed(":"), 2)[,1]) |>
  group_by(mhc.a) |>
  summarise(records = n(), publications = length(unique(reference.id))) |>
  ungroup() |>
  mutate(fraction.records = records / sum(records) * 100,
         fraction.publications = publications / sum(publications) * 100) |>
  arrange(-records)

data.s.mhc |>
  head()
```

```
## # A tibble: 6 x 5
##   mhc.a      records publications fraction.records fraction.publications
##   <chr>        <int>        <int>            <dbl>                 <dbl>
## 1 HLA-A*02     29435          138            47.2                  31.9
## 2 HLA-A*03     14438           13            23.2                   3.01
## 3 HLA-B*08      3633           23             5.83                  5.32
## 4 HLA-A*11      2875            8             4.61                  1.85
## 5 HLA-DRA*01    2487           34             3.99                  7.87
## 6 HLA-A*01      1998           17             3.21                  3.94
```

```r
data.s.mhc |>
  summarise(records.m = median(records),
            publications.m = median(publications))
```

```
## # A tibble: 1 x 2
##   records.m publications.m
##       <dbl>          <dbl>
## 1        37              2
```

```r
data.motifs.s.mhc <- data.motifs |>
  mutate(mhc.a = str_split_fixed(mhc.a, fixed(":"), 2)[,1]) |>
  group_by(gene, mhc.a) |>
  summarise(records = n(), motifs = length(unique(cid))) |>
  ungroup() |>
  mutate(fraction.records = records / sum(records) * 100,
         fraction.motifs = motifs / sum(motifs) * 100) |>
  arrange(-records)
```

```
## `summarise()` has grouped output by 'gene'. You can override using the
## `.groups` argument.
```

```r
data.motifs.s.mhc |>
  head()
```

```
## # A tibble: 6 x 6
##   gene  mhc.a      records motifs fraction.records fraction.motifs
##   <chr> <chr>        <int>  <int>            <dbl>           <dbl>
## 1 TRA   HLA-A*03      3076    207            39.5            40.8
## 2 TRB   HLA-A*02      1676     68            21.5            13.4
## 3 TRA   HLA-A*02      1389     77            17.8            15.2
## 4 TRB   HLA-A*03       785     57            10.1            11.2
## 5 TRB   HLA-DRA*01     119     10             1.53            1.97
## 6 TRA   HLA-DRA*01     105      5             1.35            0.986
```

```r
data.motifs.s.mhc |>
  group_by(gene) |>
  summarise(records.m = median(records),
            motifs.m = median(motifs))
```

```
## # A tibble: 2 x 3
##   gene  records.m motifs.m
##   <chr>     <int>    <int>
## 1 TRA          46        5
## 2 TRB          32        5
```

By epitope

```r
data.s.ag <- data |>
  group_by(antigen.epitope) |>
  summarise(records = n(), publications = length(unique(reference.id))) |>
  ungroup() |>
  mutate(fraction.records = records / sum(records) * 100,
         fraction.publications = publications / sum(publications) * 100) |>
  arrange(-records)

data.s.ag |>
  head()
```

```
## # A tibble: 6 x 5
##   antigen.epitope records publications fraction.records fraction.publications
```

```
##   <chr>              <int>        <int>            <dbl>                  <dbl>
## 1 KLGGALQAK          13698            1             22.0                   0.0616
## 2 GILGFVFTL          10373           21             16.6                   1.29
## 3 NLVPMVATV           7044           31             11.3                   1.91
## 4 RAKFKQLL            2251            6              3.61                  0.370
## 5 AVFDRKSDAK          1739            4              2.79                  0.246
## 6 GLCTLVAML           1663           23              2.67                  1.42
```

```
data.s.ag |>
  summarise(records.m = median(records),
            publications.m = median(publications))
```

```
## # A tibble: 1 x 2
##   records.m publications.m
##       <dbl>          <dbl>
## 1         2              1
```

```
data.motifs.s.ag <- data.motifs |>
  group_by(gene, antigen.epitope) |>
  summarise(records = n(), motifs = length(unique(cid))) |>
  ungroup() |>
  mutate(fraction.records = records / sum(records) * 100,
         fraction.motifs = motifs / sum(motifs) * 100) |>
  arrange(-records)
```

```
## `summarise()` has grouped output by 'gene'. You can override using the
## `.groups` argument.
```

```
data.motifs.s.ag |>
  head()
```

```
## # A tibble: 6 x 6
##   gene  antigen.epitope records motifs fraction.records fraction.motifs
##   <chr> <chr>             <int>  <int>            <dbl>           <dbl>
## 1 TRA   KLGGALQAK          3068    206            39.4            40.6
## 2 TRB   GILGFVFTL           953     18            12.2             3.55
## 3 TRA   GILGFVFTL           847     32            10.9             6.31
## 4 TRB   KLGGALQAK           785     57            10.1            11.2
## 5 TRB   NLVPMVATV           263     25             3.38            4.93
## 6 TRB   YLQPRTFLL           262      5             3.36            0.986
```

```
data.motifs.s.ag |>
  group_by(gene) |>
  summarise(records.m = median(records),
            motifs.m = median(motifs))
```

```
## # A tibble: 2 x 3
##   gene  records.m motifs.m
##   <chr>     <int>    <int>
## 1 TRA          16        3
## 2 TRB          11        1
```

Overview of VDJdb confidence scoring features

By assay type

```
method_variants <- summary(as.factor(data$method.identification))
tibble(method = names(method_variants), records = method_variants) |>
  arrange(-records)
```

```
## # A tibble: 49 x 2
##    method                                                  records
##    <chr>                                                     <int>
##  1 dextramer-sort                                            22993
##  2 tetramer-sort                                             21977
##  3 antigen-loaded-targets,dextramer-sort                      9932
##  4 peptide-stimulation, Cultured-T-cells, tetramer-sort       1511
##  5 cultured-T-cells,tetramer-sort                             1112
##  6 multimer-sort                                               909
##  7 antigen-loaded-targets                                      704
##  8 cultured-T-cells                                            667
##  9 NA's                                                        392
## 10 tetramer.sort                                               273
## # i 39 more rows
```

```
summary(as.factor(data$method.singlecell))
```

```
##          no Single cell         yes        NA's
##          11        1105       37004       24213
```

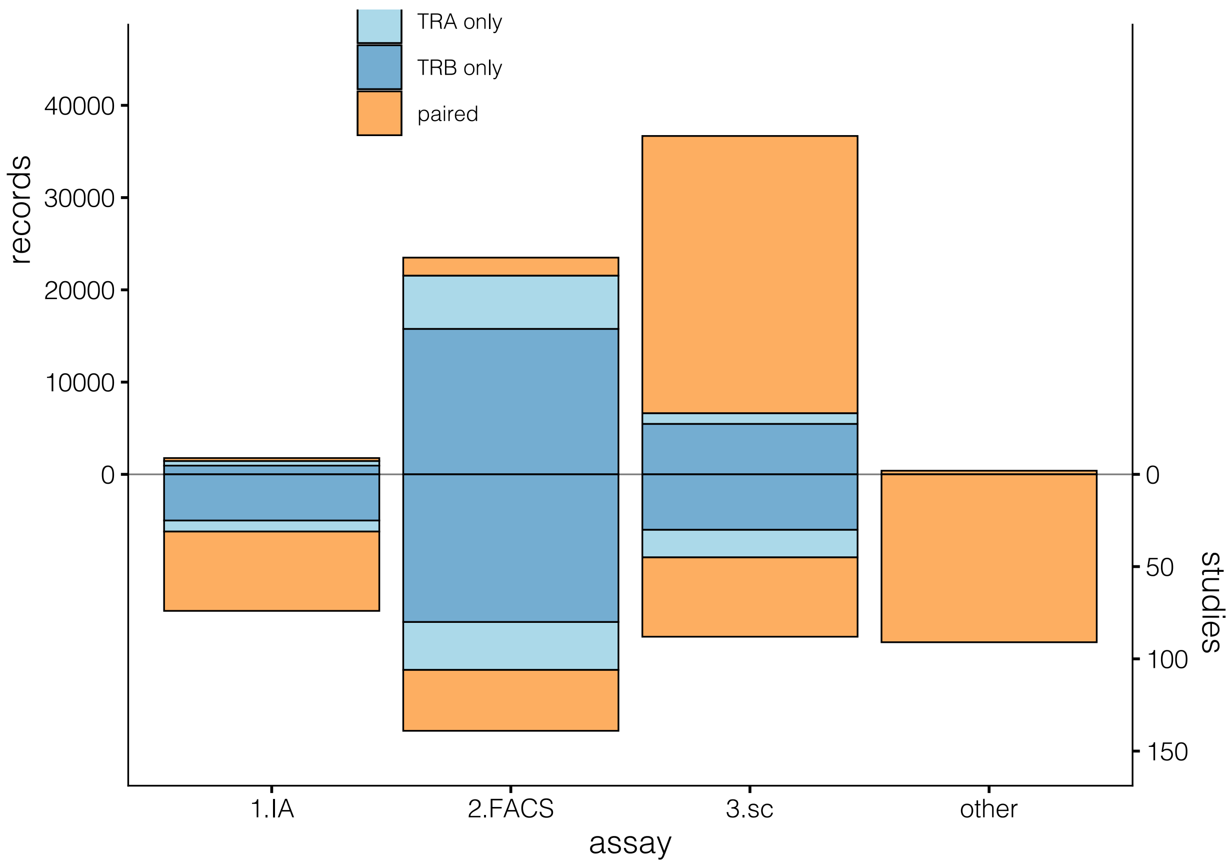
```
data.assay <- data |>
  mutate(assay = case_when(method.singlecell == "yes" &
                             grepl("sort", method.identification) ~ '3.scRNA-seq',
                           (is.na(method.singlecell) | method.singlecell != "yes") &
                             grepl("sort", method.identification) ~ '2.MHC-multimer FACS',
                           (!is.na(method.identification) &
                             !grepl("sort", method.identification)) ~ '1.immunoassay',
                           T ~ 'other'),
         chain = case_when(is.na(cdr3.beta) ~ "TRA only",
                           is.na(cdr3.alpha) ~ "TRB only",
                           !is.na(cdr3.beta) & !is.na(cdr3.alpha) ~ "paired"))

data.assay.s <- data.assay |>
  group_by(chain, assay) |>
  summarise(records = n(), studies = length(unique(reference.id)))
```

```
## `summarise()` has grouped output by 'chain'. You can override using the
## `.groups` argument.
```

```
coef = 200
brks = c(0, 10000, 20000, 30000, 40000, 50000, 60000)
p1 <- ggplot(data.assay.s, aes(x = assay,
                         fill = chain)) +
```

```r
  geom_bar(aes(y = records),
           stat="identity",
           color = "black", linewidth = 0.3) +
  geom_bar(aes(y = -studies*coef),
           stat="identity",
           color = "black", linewidth = 0.3) +
  geom_hline(yintercept = 0,
             alpha = 0.5,
             linewidth = 0.3) +
  scale_y_continuous(
    # Features of the first axis
    name = "records",
    breaks = brks,
    limits = c(-30000, 45000),
    # Add a second axis and specify its features
    sec.axis = sec_axis(~.*-coef, name="studies",
                        breaks = brks * coef,
                        labels = brks / coef)
  ) +
  scale_fill_manual("",
                    breaks = c("TRA only",
                               "TRB only",
                               "paired"),
                    values = c("#abd9e9", "#74add1", "#fdae61")) +
  scale_x_discrete("assay", labels = c("1.IA", "2.FACS", "3.sc", "other")) +
  theme_classic() +
  theme(
    text=element_text(size=12, family="Helvetica light"),
    axis.line = element_line(color='black', linewidth = 0.3),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.title.y = element_text(hjust=0.8),
    legend.background = element_blank(),
    legend.position = c(0.30, 0.97),
    legend.text = element_text(size=8),
    legend.title = element_text(size=10)#,
    #axis.text.x = element_text(angle=90, vjust=.5, hjust=1)
    )
p1
```



By number of studies

```
data.slim <- read_tsv("dump/vdjdb-2025-02-21/vdjdb.slim.txt")
```

```
## Rows: 85008 Columns: 16
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (13): gene, cdr3, species, antigen.epitope, antigen.gene, antigen.specie...
## dbl  (1): vdjdb.score
## num  (2): j.start, v.end
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
glimpse(data.slim)
```

```
## Rows: 85,008
## Columns: 16
## $ gene            <chr> "TRA", "TRA", "TRA", "TRA", "TRA", "TRA", "TRA", "TRA"~
## $ cdr3            <chr> "CAAAAGNTGKLIF", "CAAAAGNTGKLIF", "CAAAAPSSDKVIF", "CA~
## $ species         <chr> "HomoSapiens", "MusMusculus", "HomoSapiens", "HomoSapi~
## $ antigen.epitope <chr> "SLFFSAQPFEITAST", "HGIRNASFI", "RAKFKQLL", "EPLPQGQLT~
## $ antigen.gene    <chr> "APOB", "M45", "BZLF1", "BZLF1", "GAD2", "M", "MART1",~
## $ antigen.species <chr> "HomoSapiens", "MCMV", "EBV", "EBV", "HomoSapiens", "I~
## $ complex.id      <chr> "44680", "3387", "40451", "38479", "41248", "38092", "~
```

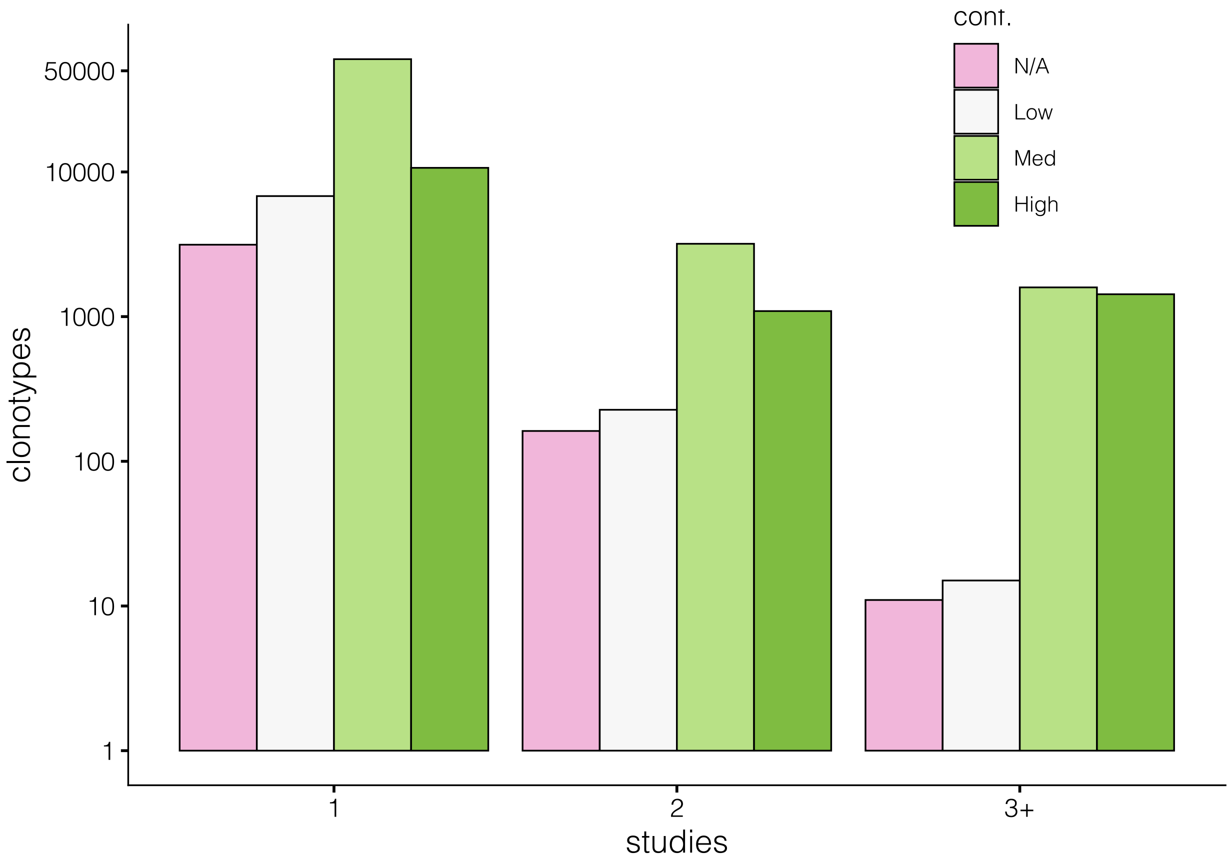
```
## $ v.segm          <chr> "TRAV13-1*01", "TRAV5N-4*01", "TRAV13-1*01", "TRAV1-2*~
## $ j.segm          <chr> "TRAJ37*01", "TRAJ37*01", "TRAJ50*01", "TRAJ6*01", "TR~
## $ mhc.a           <chr> "HLA-DRA*01:01:01", "H-2Db", "HLA-B*08:01", "HLA-B*35:~
## $ mhc.b           <chr> "HLA-DRB1*07:01", "B2M", "B2M", "B2M", "B2M", "B2M", "~
## $ mhc.class       <chr> "MHCII", "MHCI", "MHCI", "MHCI", "MHCI", "MHCI", "MHCI~
## $ reference.id    <chr> "PMID:35990517", "PMID:28636592", "PMID:34811538", "PM~
## $ vdjdb.score     <dbl> 2, 3, 1, 2, 1, 1, 0, 2, 2, 2, 2, 2, 2, 1, 1, 2, 2, 2, ~
## $ j.start         <dbl> 5, 5, 9, 4, 8, 7, 8, 5, 5, 4, 4, 4, 4, 7, 8, 5, 5, 5, ~
## $ v.end           <dbl> 3, 3, 3, 2, 2, 2, -1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3,~
```

```
data.score <- data.slim |>
  separate_rows(reference.id, sep = ",") |>
  group_by(gene, v.segm, j.segm, cdr3) |>
  mutate(studies = length(unique(reference.id))) |>
  mutate(studies = ifelse(studies > 2, "3+", as.character(studies))) |>
  group_by(vdjdb.score, studies) |>
  summarise(clonotypes = n())
```

```
## `summarise()` has grouped output by 'vdjdb.score'. You can override using the
## `.groups` argument.
```

```
p1a <- ggplot(data.score |>
         mutate(vdjdb.score = case_when(
           vdjdb.score == 1 ~ "Low",
           vdjdb.score == 2 ~ "Med",
           vdjdb.score == 3 ~ "High",
           T ~ "N/A"
         ) |> factor(levels = c("N/A", "Low", "Med", "High"))),
       aes(x = studies, y = clonotypes, fill = vdjdb.score)) +
  geom_bar(stat = "identity", position = "dodge", color = "black", linewidth = 0.3) +
  #facet_wrap(~ vdjdb.score, ncol = 1, strip.position = "left", dir = "v") +
  scale_y_log10(breaks = c(1, 10, 100, 1000, 10000, 50000)) +
  scale_fill_manual("VDJdb\nconf.",
                    breaks = c("N/A",
                               "Low",
                               "Med",
                               "High"),
                    values = c("#f1b6da", "#f7f7f7", "#b8e186", "#7fbc41")) +
  theme_classic() +
  theme(
    text = element_text(size=12,  family="Helvetica light"),
    axis.line = element_line(color='black', linewidth = 0.3),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    #axis.title.y = element_text(hjust=0.8),
    legend.background = element_blank(),
    legend.position = c(0.8, 0.9),
    legend.text = element_text(size=8),
    legend.title = element_text(size=10)#,
    #axis.text.x = element_text(angle=90, vjust=.5, hjust=1)
    )

p1a
```



More detailed view of studies per clonotype by assay

```
data.epi <- rbind(data.assay |>
                        filter(!is.na(cdr3.beta)) |>
                        group_by(cdr3.beta) |>
                        mutate(
                          chain = "TRB",
                          epitopes = length(unique(antigen.epitope))),
                      data.assay |>
                        filter(!is.na(cdr3.alpha)) |>
                        group_by(cdr3.alpha) |>
                        mutate(
                          chain = "TRA",
                          epitopes = length(unique(antigen.epitope))))|>
  group_by(assay, chain, epitopes) |>
  summarise(clonotypes = length(unique(paste(v.alpha, cdr3.alpha, j.alpha,
                                             v.beta, cdr3.beta, j.beta))))
```

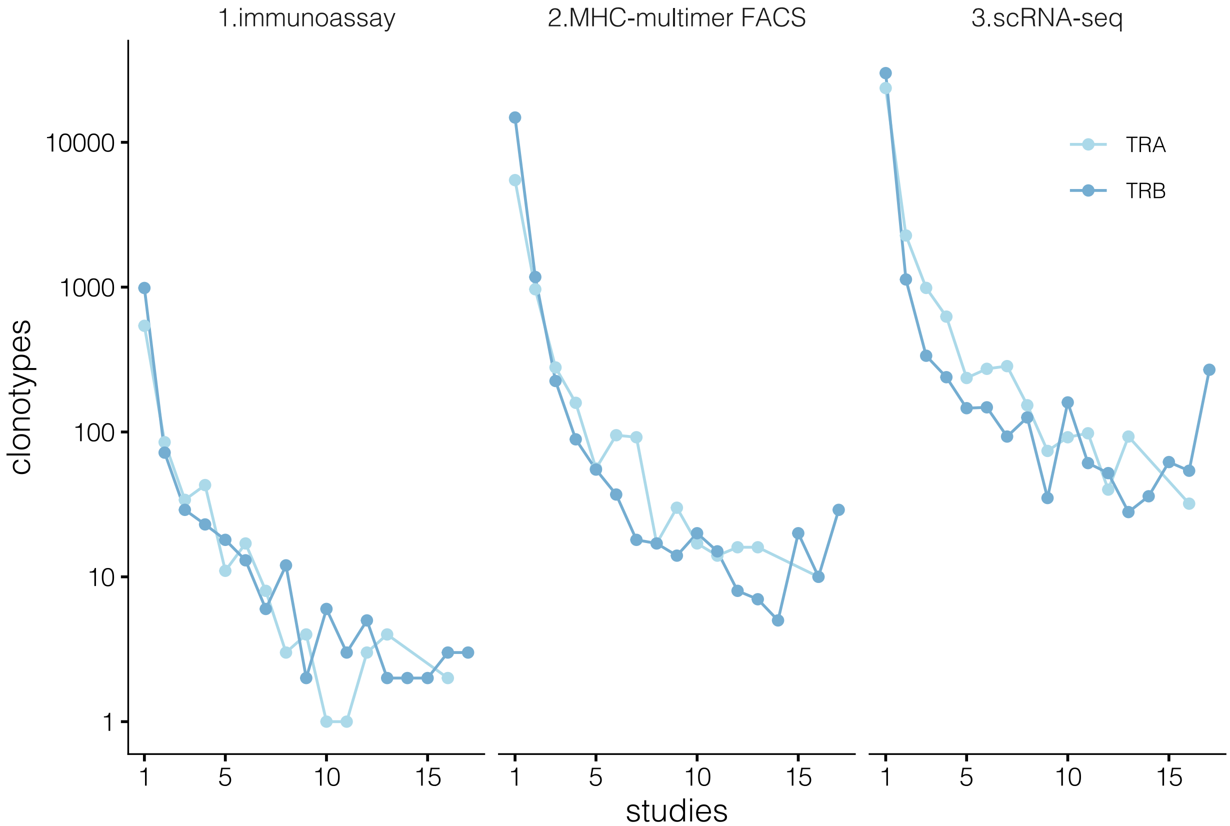
```
## `summarise()` has grouped output by 'assay', 'chain'. You can override using
## the `.groups` argument.
```

```
data.studies <- rbind(data.assay |>
                        filter(!is.na(cdr3.beta)) |>
                        group_by(cdr3.beta) |>
                        mutate(
                          chain = "TRB",
```

```
                          studies = length(unique(reference.id))),
                      data.assay |>
                        filter(!is.na(cdr3.alpha)) |>
                        group_by(cdr3.alpha) |>
                        mutate(
                          chain = "TRA",
                          studies = length(unique(reference.id))))|>
  group_by(assay, chain, studies) |>
  summarise(clonotypes = length(unique(paste(v.alpha, cdr3.alpha, j.alpha,
                                              v.beta, cdr3.beta, j.beta))))
```

```
## ‘summarise()‘ has grouped output by ’assay’, ’chain’. You can override using
## the ‘.groups‘ argument.
```

```
p2 <- ggplot(data.studies |>
               filter(assay != "other"),
             aes(x = studies, y = clonotypes, color = chain)) +
  geom_path() +
  geom_point() +
  scale_y_log10() +
  scale_x_continuous(breaks = c(1, 5, 10, 15)) +
  scale_color_manual("",
                     breaks = c("TRA", "TRB"),
                     values = c("#abd9e9", "#74add1")) +
  facet_wrap(~assay) +
  theme_classic() +
  theme(
    text=element_text(size=12,  family="Helvetica light"),
    axis.line = element_line(color='black', linewidth = 0.3),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    legend.background = element_blank(),
    strip.background = element_blank(),
    legend.position = c(0.90, 0.85),
    legend.text = element_text(size=8),
    legend.title = element_text(size=10))
p2
```

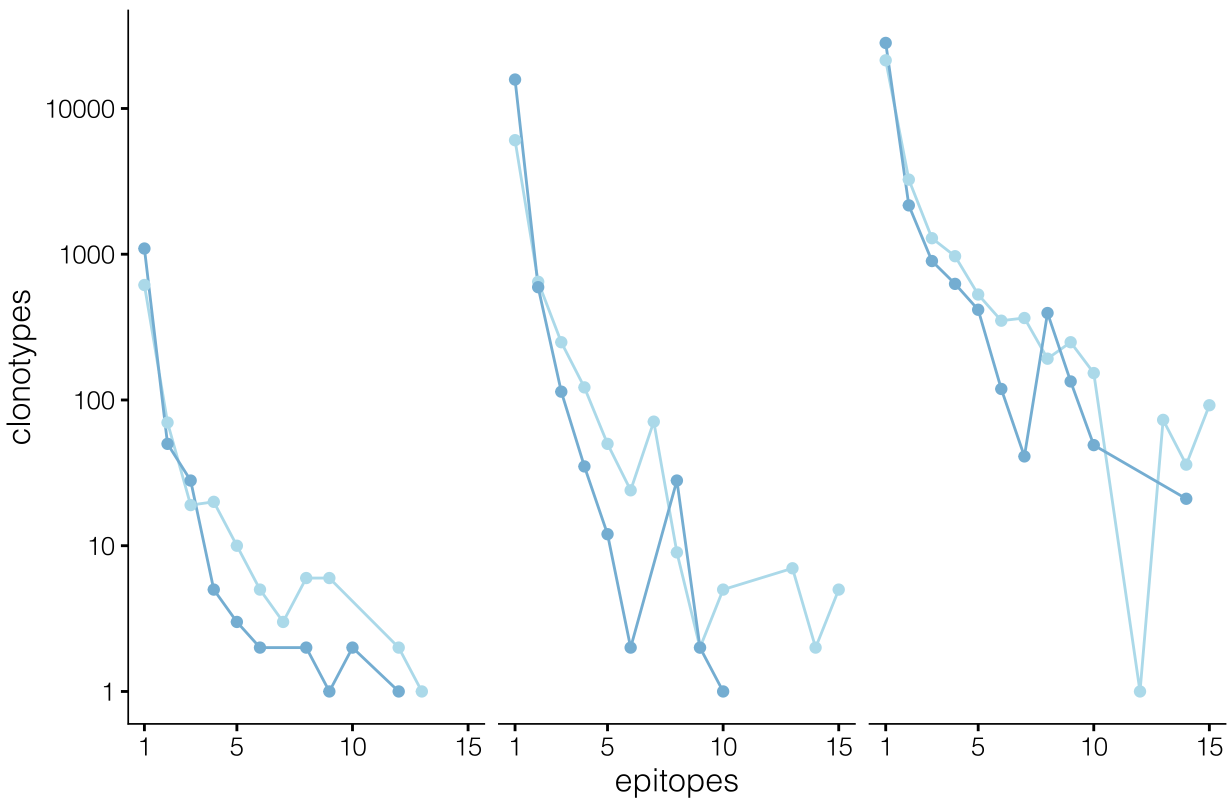



```r
p3 <- ggplot(data.epi |>
               filter(assay != "other"),
             aes(x = epitopes, y = clonotypes, color = chain)) +
  geom_path() +
  geom_point() +
  scale_y_log10() +
  scale_x_continuous(breaks = c(1, 5, 10, 15)) +
  scale_color_manual(guide = F,
                     breaks = c("TRA", "TRB"),
                     values = c("#abd9e9", "#74add1")) +
  facet_wrap(~assay) +
  theme_classic() +
  theme(
    text=element_text(size=12,  family="Helvetica light"),
    axis.line = element_line(color='black', linewidth = 0.3),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    strip.background = element_rect(size = 0.5, fill = 'white', color = "white"),
    strip.text = element_text(colour = 'white'))
```

```
## Warning: The `size` argument of `element_rect()` is deprecated as of ggplot2 3.4.0.
## i Please use the `linewidth` argument instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
p3
```

```
## Warning: The `guide` argument in `scale_*()` cannot be `FALSE`. This was deprecated in
## ggplot2 3.3.4.
## i Please use "none" instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Number of supporting studies for records that fall into motifs

```
data.motifs.1 <- rbind(data.assay |>
                     filter(!is.na(cdr3.beta)) |>
                     mutate(in.motif = cdr3.beta %in% data.motifs$cdr3aa) |>
                     group_by(cdr3.beta) |>
                     mutate(
                       chain = "TRB",
                       epitopes = length(unique(antigen.epitope)),
                       studies = length(unique(reference.id))),
                   data.assay |>
                     filter(!is.na(cdr3.alpha)) |>
                     mutate(in.motif = cdr3.alpha %in% data.motifs$cdr3aa) |>
                     group_by(cdr3.alpha) |>
                     mutate(
                       chain = "TRA",
                       epitopes = length(unique(antigen.epitope)),
                       studies = length(unique(reference.id)))) |>
```

```r
  group_by(chain, in.motif, epitopes, studies) |>
  summarise(clonotypes = length(unique(paste(v.alpha, cdr3.alpha, j.alpha,
                                             v.beta, cdr3.beta, j.beta)))) |>
  group_by(chain, in.motif) |>
  mutate(total_clonotypes = sum(clonotypes)) |>
  ungroup()
```

```
## `summarise()` has grouped output by 'chain', 'in.motif', 'epitopes'. You can
## override using the `.groups` argument.
```

```r
data.motifs.1
```

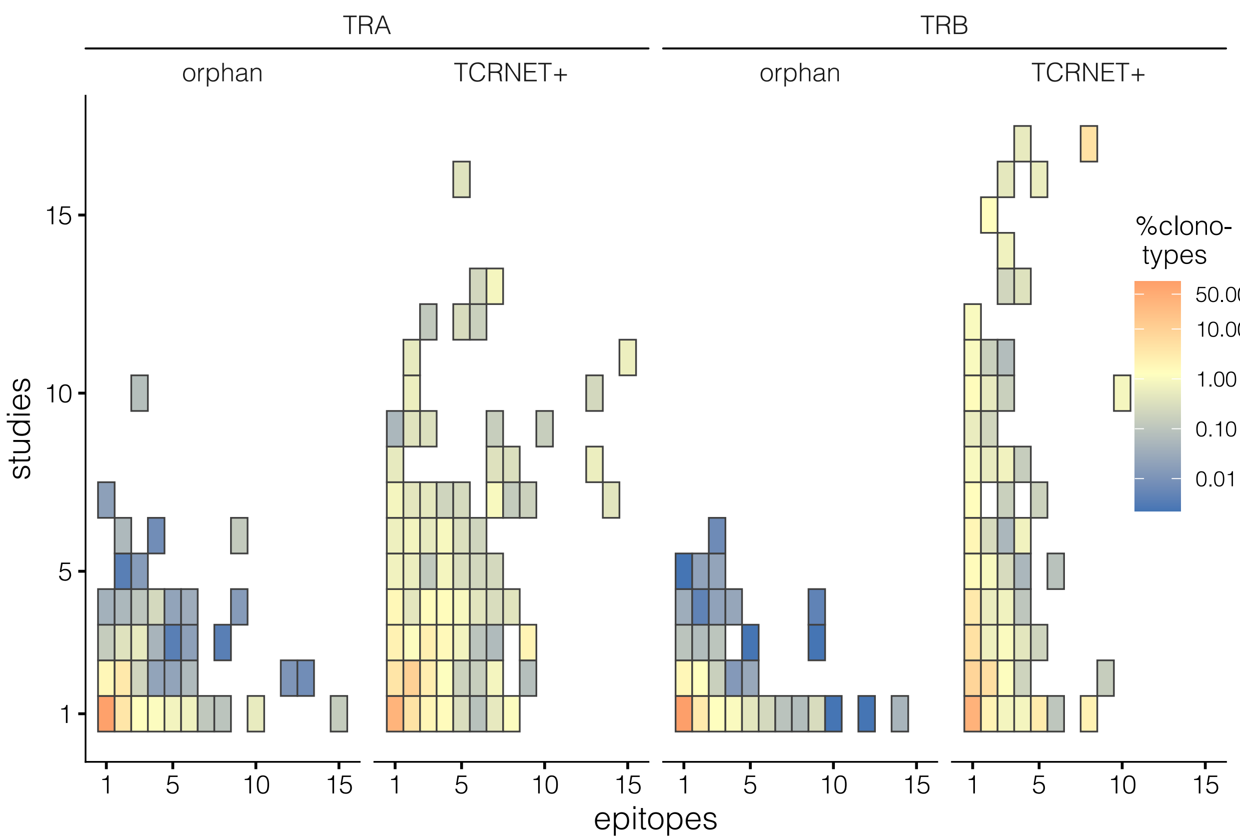
```
## # A tibble: 198 x 6
##    chain in.motif epitopes studies clonotypes total_clonotypes
##    <chr> <lgl>       <int>   <int>      <int>            <int>
##  1 TRA   FALSE           1       1      23125            27531
##  2 TRA   FALSE           1       2        489            27531
##  3 TRA   FALSE           1       3         38            27531
##  4 TRA   FALSE           1       4         11            27531
##  5 TRA   FALSE           1       7          5            27531
##  6 TRA   FALSE           2       1       1048            27531
##  7 TRA   FALSE           2       2        932            27531
##  8 TRA   FALSE           2       3        100            27531
##  9 TRA   FALSE           2       4         15            27531
## 10 TRA   FALSE           2       5          1            27531
## # i 188 more rows
```

```r
p4 <- ggplot(data.motifs.1 |>
               mutate(in.motif = ifelse(in.motif, "TCRNET+", "orphan")),
             aes(x = epitopes, y = studies)) +
  geom_tile(color = "grey25", size = 0.3,
            #binwidth = c(1, 1),
            aes(fill = clonotypes / total_clonotypes * 100)) +
  scale_fill_gradient2("%clono-\n types",
                       trans = "log10",
                       mid = "#ffffbf", low = "#4575b4", high = "#f46d43",
                       breaks = c(0.01, 0.1, 1, 10, 50),
                       midpoint = 0.1) +
  scale_x_continuous(breaks = c(1, 5, 10, 15)) +
  scale_y_continuous(breaks = c(1, 5, 10, 15)) +
  #scale_y_continuous(limits = c(0, 17), breaks = c(1, 5, 10, 15)) +
  #scale_y_continuous(limits = c(0, 17), breaks = c(1, 5, 10, 15), expand = expansion(0, 1)) +
  #facet_grid(~ chain + in.motif) +
  facet_nested( ~ chain + in.motif, nest_line = T) +
  theme_classic() +
  theme(
    #aspect.ratio = 1,
    text = element_text(size=12,  family="Helvetica light"),
    axis.line = element_line(color = 'black', linewidth = 0.3),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    strip.background = element_blank(),
```

```
    legend.background = element_blank(),
    legend.position = c(0.97, 0.6),
    legend.text = element_text(size=8),
    legend.title = element_text(size=10)
    #strip.background = element_rect(size = 0.5)
    )
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
p4
```



Matchmakers data analysis

First, just estimate using numbers from Messemaker et al. We don't know outcome for each record, just use reported validation rates as priors.

```
data.slim |>
  filter(antigen.epitope %in% c("YLQPRTFLL", "GLCTLVAML"),
         complex.id != 0) |>
  mutate(in.motif = ifelse(cdr3 %in% data.motifs$cdr3aa,
                           "TCRNET+", "orphan")) |>
```

```r
  mutate(vdjdb.score = case_when(
    vdjdb.score == 1 ~ "Lo",
    vdjdb.score == 2 ~ "Med",
    vdjdb.score == 3 ~ "Hi",
    T ~ "N/A"
  ) |> factor(levels = c("N/A", "Lo", "Med", "Hi"))) |>
  separate_rows(reference.id, sep = ",") |>
  group_by(gene, v.segm, j.segm, cdr3) |>
  mutate(studies = length(unique(reference.id))) |>
  mutate(studies = ifelse(studies > 2, "3+", as.character(studies))) |>
  group_by(antigen.epitope, vdjdb.score, studies, in.motif) |>
  summarise(clonotypes = length(unique(paste(v.segm, j.segm, cdr3)))) -> data.alluvial
```

```
## 'summarise()' has grouped output by 'antigen.epitope', 'vdjdb.score',
## 'studies'. You can override using the '.groups' argument.
```

```r
# Matchmakers estimate
tibble(antigen.epitope = c("YLQPRTFLL", "GLCTLVAML", "YLQPRTFLL", "GLCTLVAML"),
       matchmakers = c("MM+", "MM+", "MM-", "MM-"),
       mm_frac_ae = c(0.406, 0.694, 1 - 0.406, 1 - 0.694)) |>
  full_join(tibble(vdjdb.score = c("N/A", "Lo", "Med", "Hi", "N/A", "Lo", "Med", "Hi"),
                   matchmakers = c("MM+", "MM+", "MM+", "MM+", "MM-", "MM-", "MM-", "MM-"),
                   mm_frac_vs = c(0.65, 0.45, 0.8, 1.0, 1.0 - 0.65, 1.0 - 0.45, 1.0 - 0.8, 1.0 - 1.0)) |>
              group_by(matchmakers) |>
              mutate(mm_frac_vs = mm_frac_vs / sum(mm_frac_vs)),
            by = "matchmakers", relationship = "many-to-many") -> data.mm

data.alluvial.mm <- full_join(data.alluvial,
                              data.mm,
                              by = c("antigen.epitope", "vdjdb.score"),
                              relationship = "many-to-many") |>
  mutate(frac = clonotypes * mm_frac_ae * mm_frac_vs)
```

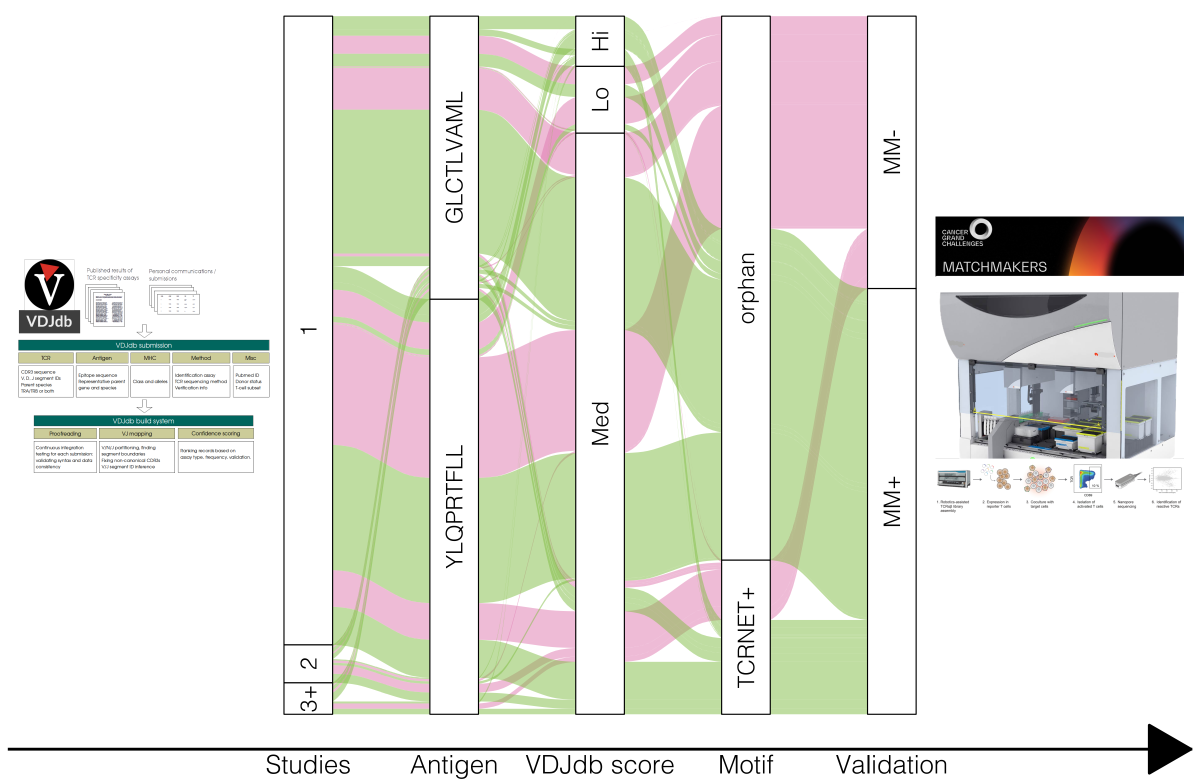
```r
img.vdjdb <- rasterGrob(readPNG("assets/vdjdb.png"), interpolate=T)
img.mm <- rasterGrob(readPNG("assets/mm.png"), interpolate=T)

p5 <- ggplot(data = data.alluvial.mm |>
         filter(vdjdb.score != "N/A") |>
         group_by(antigen.epitope, studies,
                  vdjdb.score, in.motif, matchmakers) |>
         summarise(frac = sum(frac)),
       aes(axis1 = studies,
           axis2 = antigen.epitope,
           axis4 = vdjdb.score,
           axis5 = in.motif,
           axis6 = matchmakers,
           y = frac)) +
  scale_x_discrete("",
                   limits = c("Studies", "Antigen", "VDJdb score", "Motif", "Validation"),
                   expand = c(.5, .05),
                   position = "bottom") +
  ylab("") +
  geom_alluvium(aes(fill = matchmakers), color = NA) +
```

```
  scale_fill_manual(guide = F, values = c("#de77ae", "#7fbc41")) +
  geom_stratum(color = "black", fill = "white", size = 0.2) +
  geom_text(stat = "stratum",
            aes(label = after_stat(stratum)),
            angle = 90, family="Helvetica light",
            cex = 3) +
  annotation_custom(img.vdjdb, xmin=-1.0, xmax=0.75, ymin=-Inf, ymax=Inf) +
  annotation_custom(img.mm, xmin=5.3, xmax=7, ymin=-Inf, ymax=Inf) +
  theme_classic() +
  theme(text = element_text(size=12, family="Helvetica light"),
        axis.line.y = element_blank(),
        axis.line.x.bottom = element_line(arrow = arrow(type="closed")),
        axis.text.y = element_blank(),
        axis.ticks = element_blank(),
        legend.position = "bottom")
```

```
## `summarise()` has grouped output by 'antigen.epitope', 'studies',
## 'vdjdb.score', 'in.motif'. You can override using the `.groups` argument.
```

```
p5
```



Use Matchmakers table, i.e. outcomes for each record. N.B. The actual table is not present in the repo, one should write to the authors to obtain a copy.

```
data.mm <- read_csv("matchmakers/01_05_2025_TCRvdb.csv")
```

```
## New names:
```

```
## Rows: 3693 Columns: 26
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (17): name, clonotype_aa, cdr3_alpha_aa, cdr3_beta_aa, TRAV, TRAJ, TRBV,... dbl
## (8): ...1, vdjdb_score, baseMean, log2FoldChange, lfcSE, stat, pvalue, ... lgl
## (1): TRBD_IMGT
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```
data.mm <- rbind(tibble(cdr3 = data.mm$cdr3_alpha_aa,
                        v.segm = data.mm$TRAV,
                        padj = data.mm$padj),
                 tibble(cdr3 = data.mm$cdr3_beta_aa,
                        v.segm = data.mm$TRBV,
                        padj = data.mm$padj)) |>
  group_by(cdr3) |>
  summarise(padj = min(padj))

data.slim |>
  filter(antigen.epitope %in% c("YLQPRTFLL", "GLCTLVAML"),
         complex.id != 0) |> # paired records only
  mutate(in.motif = ifelse(cdr3 %in% data.motifs$cdr3aa,
                           "TCRNET+", "orphan")) |>
  mutate(vdjdb.score = case_when(
    vdjdb.score == 1 ~ "Lo",
    vdjdb.score == 2 ~ "Med",
    vdjdb.score == 3 ~ "Hi",
    T ~ "?"
  ) |> factor(levels = c("?", "Lo", "Med", "Hi"))) |>
  separate_rows(reference.id, sep = ",") |>
  group_by(gene, v.segm, j.segm, cdr3) |>
  mutate(studies = length(unique(reference.id))) |>
  mutate(studies = ifelse(studies > 2, "3+", as.character(studies))) |>
  left_join(data.mm) |>
  mutate(matchmakers = ifelse(is.na(padj), "?", ifelse(padj < 1e-5, "MM+", "MM-"))) |>
  group_by(antigen.epitope, vdjdb.score, studies, in.motif, matchmakers) |>
  summarise(clonotypes = length(unique(paste(v.segm, j.segm, cdr3)))) -> data.alluvial.mm.2
```

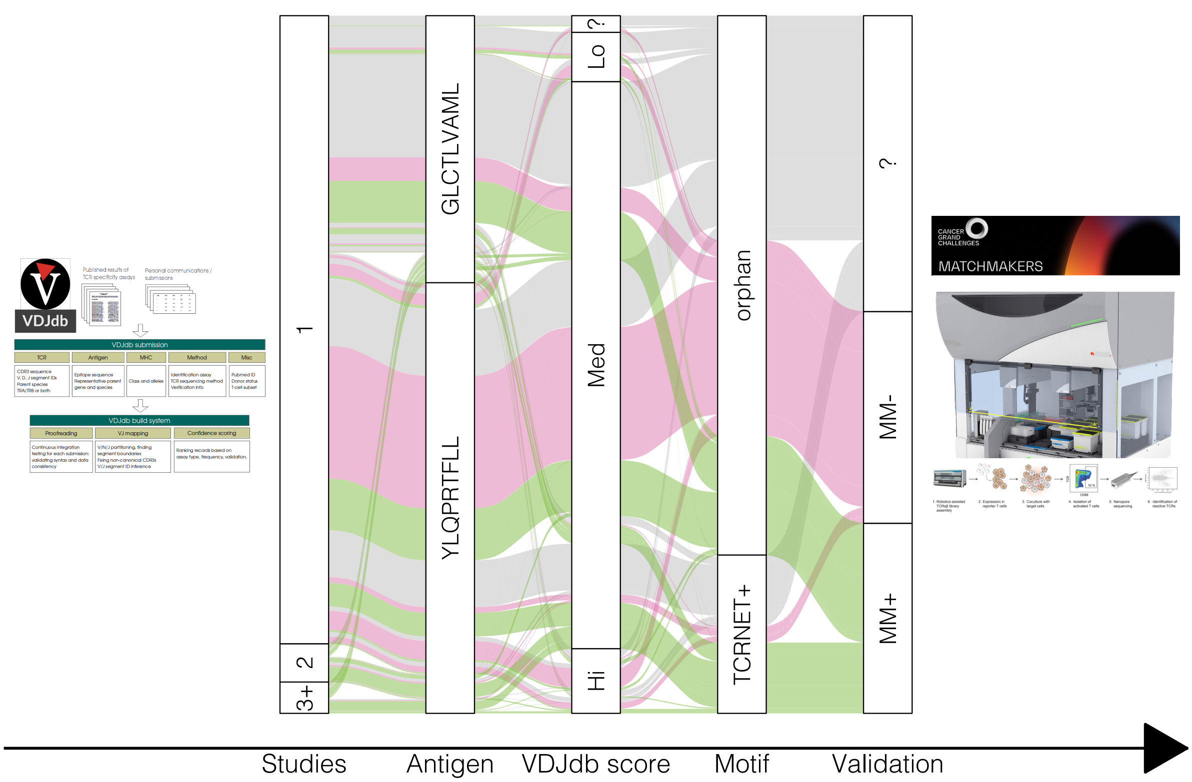
```
## Joining with `by = join_by(cdr3)`
## `summarise()` has grouped output by 'antigen.epitope', 'vdjdb.score',
## 'studies', 'in.motif'. You can override using the `.groups` argument.
```

```
p6 <- ggplot(data = data.alluvial.mm.2,# |>
        # filter(vdjdb.score != "N/A"),
       aes(axis1 = studies,
           axis2 = antigen.epitope,
           axis4 = vdjdb.score,
           axis5 = in.motif,
           axis6 = matchmakers,
           y = clonotypes)) +
  scale_x_discrete("",
                   limits = c("Studies", "Antigen", "VDJdb score", "Motif", "Validation"),
```

```
                     expand = c(.5, .05),
                     position = "bottom") +
  ylab("") +
  geom_alluvium(aes(fill = matchmakers), color = NA) +
  scale_fill_manual(guide = F, values = c("grey", "#de77ae", "#7fbc41")) +
  geom_stratum(color = "black", fill = "white", size = 0.2) +
  geom_text(stat = "stratum",
            aes(label = after_stat(stratum)),
            angle = 90, family="Helvetica light",
            cex = 3) +
  annotation_custom(img.vdjdb, xmin=-1.0, xmax=0.75, ymin=-Inf, ymax=Inf) +
  annotation_custom(img.mm, xmin=5.3, xmax=7, ymin=-Inf, ymax=Inf) +
  theme_classic() +
  theme(text = element_text(size=12, family="Helvetica light"),
        axis.line.y = element_blank(),
        axis.line.x.bottom = element_line(arrow = arrow(type="closed")),
        axis.text.y = element_blank(),
        axis.ticks = element_blank(),
        legend.position = "bottom")

p6
```

```
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
```



Enrichment test for validated records in TCRNET+

```
data.alluvial.mm.2 |>
  filter(matchmakers != "?") |>
  group_by(in.motif, matchmakers) |>
  summarise(clonotypes = sum(clonotypes)) -> data.alluvial.mm.s
```

```
## `summarise()` has grouped output by 'in.motif'. You can override using the
## `.groups` argument.
```

```
data.alluvial.mm.s
```

```
## # A tibble: 4 x 3
## # Groups:   in.motif [2]
##   in.motif matchmakers clonotypes
##   <chr>    <chr>            <int>
## 1 TCRNET+  MM+                174
## 2 TCRNET+  MM-                 40
## 3 orphan   MM+                293
## 4 orphan   MM-                480
```

```
dcast(data.alluvial.mm.s, in.motif ~ matchmakers) |>
  column_to_rownames("in.motif") |>
  fisher.test() -> odds
```

```
## Using clonotypes as value column: use value.var to override.
```

```
data.alluvial.mm.s$clonotypes[1] / data.alluvial.mm.s$clonotypes[2] /
  (data.alluvial.mm.s$clonotypes[3] / data.alluvial.mm.s$clonotypes[4])
```

```
## [1] 7.12628
```

```
odds
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  column_to_rownames(dcast(data.alluvial.mm.s, in.motif ~ matchmakers), "in.motif")
## p-value < 2.2e-16
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##   4.855032 10.615462
## sample estimates:
## odds ratio
##   7.111647
```

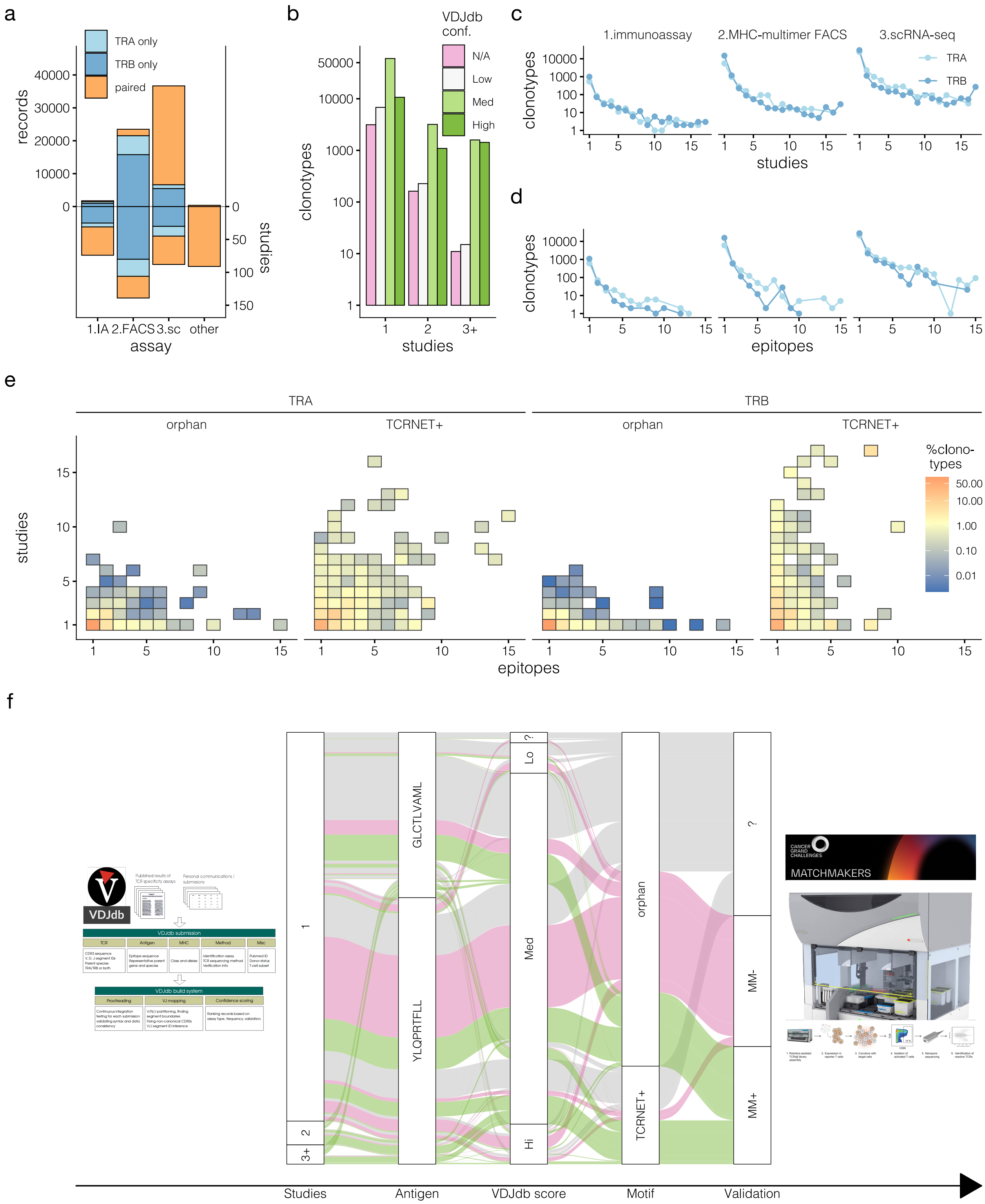
```
odds$p.value
```

```
## [1] 2.049452e-30
```

```r
tmp1 <- ((p1 + p1a + (p2 / p3)) + plot_layout(widths = c(1.1, 1.00, 2.93)))
tmp2 <- tmp1 / p4 / p6
tmp2 +
  plot_layout(heights = c(2.0, 1.5, 3.5)) +
  plot_annotation(tag_levels = 'a') &
  theme(plot.tag = element_text(size = 16, family = "Helvetica")) -> p_full

p_full
```

```
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
```




```
ggsave("figures/fig2.pdf", p_full, width = 10.5, height = 13)
```

```
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
```

```
## params$discern): Some strata appear at multiple axes.
## Warning in to_lodes_form(data = data, axes = axis_ind, discern =
## params$discern): Some strata appear at multiple axes.
```

Considering the effect of V allele calling and pairing

Checking for ambiguity in V gene assignment

```r
data |>
  group_by(cdr3.beta) |>
  summarise(distinct.v.beta.vdjdb = length(unique(v.beta))) |>
  mutate(distinct.v.beta.vdjdb = ifelse(distinct.v.beta.vdjdb >= 2,
                                        "2+", distinct.v.beta.vdjdb)) |>
  group_by(distinct.v.beta.vdjdb) |>
  summarise(unique_cdr3b = n())
```

```
## # A tibble: 2 x 2
##   distinct.v.beta.vdjdb unique_cdr3b
##   <chr>                        <int>
## 1 1                            43378
## 2 2+                             726
```

```r
data |>
  group_by(cdr3.alpha) |>
  summarise(distinct.v.alpha.vdjdb = length(unique(v.alpha))) |>
  mutate(distinct.v.alpha.vdjdb = ifelse(distinct.v.alpha.vdjdb >= 2,
                                        "2+", distinct.v.alpha.vdjdb)) |>
  group_by(distinct.v.alpha.vdjdb) |>
  summarise(unique_cdr3a = n())
```

```
## # A tibble: 2 x 2
##   distinct.v.alpha.vdjdb unique_cdr3a
##   <chr>                         <int>
## 1 1                             27256
## 2 2+                             1236
```

```r
data.mm.1 <- read_csv("matchmakers/01_05_2025_TCRvdb.csv") |>
  mutate(matchmakers = ifelse(is.na(padj), "?", ifelse(padj < 1e-5, "MM+", "MM-"))) |>
  filter(matchmakers != "?")
```

```
## New names:
## Rows: 3693 Columns: 26
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (17): name, clonotype_aa, cdr3_alpha_aa, cdr3_beta_aa, TRAV, TRAJ, TRBV,... dbl
## (8): ...1, vdjdb_score, baseMean, log2FoldChange, lfcSE, stat, pvalue, ... lgl
## (1): TRBD_IMGT
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```r
data.mm.1 |>
  group_by(cdr3.beta = cdr3_beta_aa) |>
  summarise(distinct.v.beta.mm = length(unique(TRBV_IMGT)) > 1,
            distinct.mm.outcome = length(unique(matchmakers)) > 1) |>
  group_by(distinct.v.beta.mm, distinct.mm.outcome) |>
  summarise(unique_cdr3b = n())
```

```
## `summarise()` has grouped output by 'distinct.v.beta.mm'. You can override
## using the `.groups` argument.

## # A tibble: 4 x 3
## # Groups:   distinct.v.beta.mm [2]
##   distinct.v.beta.mm distinct.mm.outcome unique_cdr3b
##   <lgl>              <lgl>                      <int>
## 1 FALSE              FALSE                        507
## 2 FALSE              TRUE                          11
## 3 TRUE               FALSE                          7
## 4 TRUE               TRUE                           6
```

```r
data.mm.1 |>
  group_by(cdr3.alpha = cdr3_alpha_aa) |>
  summarise(distinct.v.alpha.mm = length(unique(TRAV_IMGT)) > 1,
            distinct.mm.outcome = length(unique(matchmakers)) > 1) |>
  group_by(distinct.v.alpha.mm, distinct.mm.outcome) |>
  summarise(unique_cdr3a = n())
```

```
## `summarise()` has grouped output by 'distinct.v.alpha.mm'. You can override
## using the `.groups` argument.

## # A tibble: 3 x 3
## # Groups:   distinct.v.alpha.mm [2]
##   distinct.v.alpha.mm distinct.mm.outcome unique_cdr3a
##   <lgl>               <lgl>                      <int>
## 1 FALSE               FALSE                        488
## 2 FALSE               TRUE                          12
## 3 TRUE                TRUE                           2
```

Distinct validation outcome due to pairing (CDR3 aa level)

```r
read_tsv("dump/vdjdb-2025-02-21/vdjdb_full.txt") |>
  filter(species == "HomoSapiens",
         antigen.epitope %in% c("YLQPRTFLL", "GLCTLVAML")) |>
  summarise(unpaired = sum(is.na(cdr3.beta) | is.na(cdr3.alpha)),
            paired = sum(!is.na(cdr3.beta) & !is.na(cdr3.alpha)))
```

```
## Rows: 78826 Columns: 33
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (33): cdr3.alpha, v.alpha, j.alpha, cdr3.beta, v.beta, d.beta, j.beta, s...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## # A tibble: 1 x 2
##   unpaired paired
##      <int>  <int>
## 1     3519   1306
```

```
data.mm.1 |>
  group_by(cdr3_beta_aa) |>
  summarise(distinct.alpha.pair = length(unique(cdr3_alpha_aa)) > 1,
            distinct.mm.outcome = length(unique(matchmakers)) > 1) |>
  group_by(distinct.alpha.pair, distinct.mm.outcome) |>
  summarise(unique_cdr3b = n())
```

```
## `summarise()` has grouped output by 'distinct.alpha.pair'. You can override
## using the `.groups` argument.
```

```
## # A tibble: 3 x 3
## # Groups:   distinct.alpha.pair [2]
##   distinct.alpha.pair distinct.mm.outcome unique_cdr3b
##   <lgl>               <lgl>                      <int>
## 1 FALSE               FALSE                        493
## 2 TRUE                FALSE                         21
## 3 TRUE                TRUE                          17
```

```
data.mm.1 |>
  group_by(cdr3_alpha_aa) |>
  summarise(distinct.beta.pair = length(unique(cdr3_beta_aa)) > 1,
            distinct.mm.outcome = length(unique(matchmakers)) > 1) |>
  group_by(distinct.beta.pair, distinct.mm.outcome) |>
  summarise(unique_cdr3a = n())
```

```
## `summarise()` has grouped output by 'distinct.beta.pair'. You can override
## using the `.groups` argument.
```

```
## # A tibble: 3 x 3
## # Groups:   distinct.beta.pair [2]
##   distinct.beta.pair distinct.mm.outcome unique_cdr3a
##   <lgl>              <lgl>                      <int>
## 1 FALSE              FALSE                        456
## 2 TRUE               FALSE                         32
## 3 TRUE               TRUE                          14
```

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK ve
##
```

```
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Moscow
## tzcode source: internal
##
## attached base packages:
## [1] grid      stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] reshape2_1.4.4    png_0.1-8         showtext_0.9-7    showtextdb_3.0
##  [5] sysfonts_0.8.9    patchwork_1.3.2   ggh4x_0.3.1       ggalluvial_0.12.5
##  [9] lubridate_1.9.4   forcats_1.0.0     stringr_1.5.2     dplyr_1.1.4
## [13] purrr_1.1.0       readr_2.1.5       tidyr_1.3.1       tibble_3.3.0
## [17] ggplot2_4.0.0     tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] utf8_1.2.6         generics_0.1.4     stringi_1.8.7      hms_1.1.3
##  [5] digest_0.6.37      magrittr_2.0.4     evaluate_1.0.5     timechange_0.3.0
##  [9] RColorBrewer_1.1-3 fastmap_1.2.0      plyr_1.8.9         tinytex_0.57
## [13] scales_1.4.0       textshaping_1.0.3  cli_3.6.5          crayon_1.5.3
## [17] rlang_1.1.6        bit64_4.6.0-1      withr_3.0.2        yaml_2.3.10
## [21] parallel_4.5.1     tools_4.5.1        tzdb_0.5.0         vctrs_0.6.5
## [25] R6_2.6.1           lifecycle_1.0.4    bit_4.6.0          vroom_1.6.6
## [29] ragg_1.5.0         archive_1.1.12     pkgconfig_2.0.3    pillar_1.11.1
## [33] gtable_0.3.6       glue_1.8.0         Rcpp_1.1.0         systemfonts_1.2.3
## [37] xfun_0.53          tidyselect_1.2.1   rstudioapi_0.17.1  knitr_1.50
## [41] farver_2.1.2       htmltools_0.5.8.1  labeling_0.4.3     rmarkdown_2.29
## [45] compiler_4.5.1     S7_0.2.0
```